Experiment No. 2

Name: Parth Vijay Gulhane

Roll No: 23CO043

Class: TE Comp A

Title
Implementation of A* Search Algorithm for Maze Pathfinding

Problem Statement
Design and implement the A* (A-Star) Search Algorithm to find the shortest path in a maze from a given start position to a goal position.

Theory
Introduction to A* Algorithm
The A* algorithm is an informed search algorithm used for finding the shortest path between two nodes.
It combines the advantages of:
Uniform Cost Search (considers actual cost)
Greedy Best-First Search (uses heuristic) It
is widely used in:

- Pathfinding problems
- Game development
- Navigation systems

Evaluation Function
The decision-making core of A* is:

$$f(n) = g(n) + h(n)$$ Where:

g(n) → Exact cost from start node to current node
h(n) → Estimated cost from current node to goal **f(n)** → Total estimated cost

A* always expands the node with lowest f(n).

Heuristic Function (Manhattan Distance)

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

Manhattan is :

- Suitable for grid-based movement (no diagonals)
- Simple and fast
- Admissible → never overestimates

This is why A* guarantees optimal solution

Step-by-Step Working

1. Initialize start node
2. Add start node to OPEN list
3. Repeat:
    - Pick node with lowest f(n)
    - Move it to CLOSED list
    - If goal reached → stop
    - Else explore neighbours
    - Update cost values
4. Trace back path using parent links

Time & Space Complexity

Worst case: **$O(b^d)$** where:

b = branching factor d

= depth of solution

Uses **a lot of memory**

Properties of A*

Complete **:** Always finds a solution if one exists

Optimal **:** Finds shortest path if heuristic is admissible

Efficient **:** Faster than BFS in large spaces

Real-World Applications

- GPS Navigation (Google Maps)
- Robotics path planning
- Game AI (NPC movement)
- Network routing

Comparison with Other Algorithms

Algorithm	Uses Heuristic	Optimal	Speed
BFS	No	Yes	Slow
DFS	No	No	Fast but risky
A*	Yes	Yes	Efficient

Key Concepts You MUST Understand ○

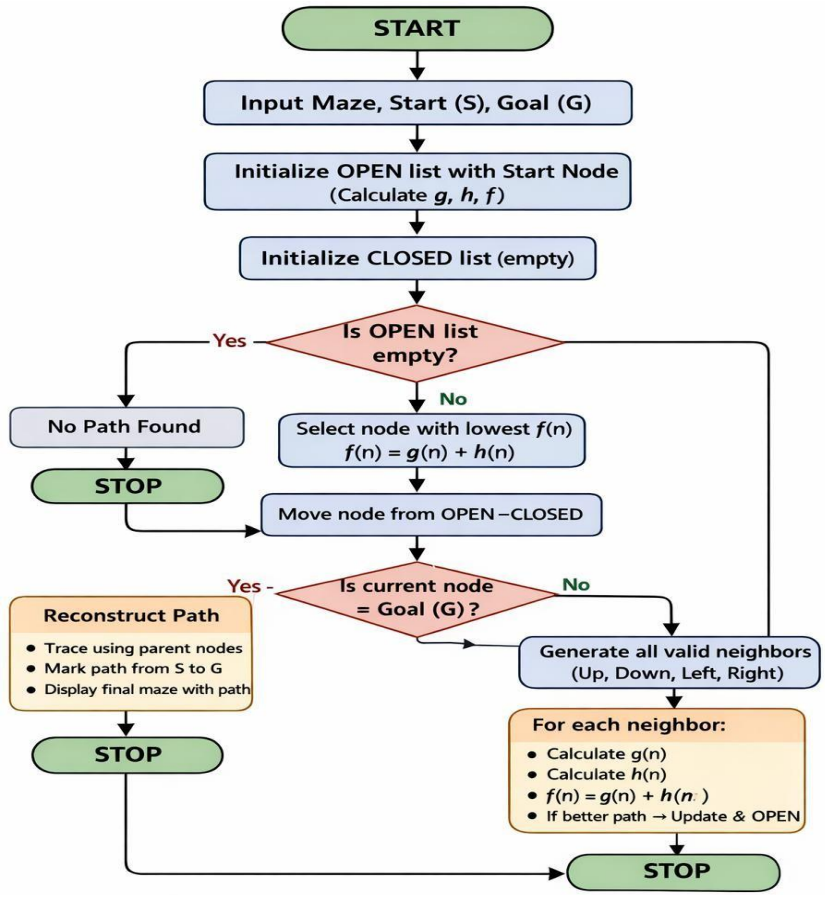
Open List

- Contains nodes yet to be explored
- Implemented using priority queue
- Sorted by lowest f(n)

- **Closed List**
  - Contains nodes already explored
  - Prevents revisiting nodes

- **Parent Tracking**
  - Each node stores its parent
  - Used to reconstruct final path

Flowchart



Output:

```
#include <iostream>
#include <vector>
#include <queue>
#include <cmath>
#include <climits>
#include <algorithm>
using namespace std;

struct Node {
    int x, y, g, h, f;
    Node(int x, int y, int g, int h) {
```

```
        this->x = x;
        this->y = y;
        this->g = g;
        this->h = h;
        this->f = g + h;
    }
    bool operator<(const Node &other) const {
        return f > other.f; // for min-heap
    }
};
// Heuristic (Manhattan Distance)
int heuristic(int x1, int y1, int x2, int y2) {
    return abs(x1 - x2) + abs(y1 - y2);
}
// Check valid cell
bool is_valid(int x, int y, int n, int m, vector<vector<int>> &maze) {
    return (x >= 0 && x < n && y >= 0 && y < m && maze[x][y] == 0);
}
// Print final path
void print_path(vector<vector<pair<int,int>>> &parent,
                pair<int,int> start,
                pair<int,int> goal) {
    vector<pair<int,int>> path;
    pair<int,int> curr = goal;
    while (curr.first != -1) {
        path.push_back(curr);
        curr = parent[curr.first][curr.second];
    }
    reverse(path.begin(), path.end());
    cout << "\nFinal Path:\n";
    for (auto &p : path) {
        cout << "(" << p.first << "," << p.second << ") ";
    }
    cout << "\nPath Length: " << path.size() - 1 << endl;
}
// A* Algorithm
void astar(vector<vector<int>> &maze,
           pair<int,int> start,
           pair<int,int> goal) {
    int n = maze.size();
    int m = maze[0].size();
    priority_queue<Node> open_list;
    open_list.push(Node(start.first, start.second, 0,
        heuristic(start.first, start.second, goal.first, goal.second)));
    vector<vector<bool>> perm_closed(n, vector<bool>(m, false));
    vector<vector<int>> g_cost(n, vector<int>(m, INT_MAX));
    vector<vector<pair<int,int>>> parent(n,
        vector<pair<int,int>>(m, {-1,-1}));
    g_cost[start.first][start.second] = 0;
    while (!open_list.empty()) {
        Node current = open_list.top();
```

```
        open_list.pop();
        int x = current.x;
        int y = current.y;
        if (perm_closed[x][y])
            continue;
        perm_closed[x][y] = true;
        cout << "\nCurrent Node: (" << x << "," << y << ") "
             << "g=" << current.g
             << " h=" << current.h
             << " f=" << current.f << endl;
        // Goal check
        if (make_pair(x,y) == goal) {
            cout << "\nGoal Reached!\n";
            print_path(parent, start, goal);
            return;
        }
        // 4 directions
        vector<pair<int,int>> directions = {
            {-1,0},{1,0},{0,-1},{0,1}
        };
        for (auto &dir : directions) {
            int nx = x + dir.first;
            int ny = y + dir.second;
            if (is_valid(nx, ny, n, m, maze)) {
                int new_g = g_cost[x][y] + 1;
                int h = heuristic(nx, ny, goal.first, goal.second);
                int f = new_g + h;
                cout << "Checking (" << nx << "," << ny << ") -> "
                     << "g=" << new_g
                     << " h=" << h
                     << " f=" << f;
                if (new_g < g_cost[nx][ny]) {
                    g_cost[nx][ny] = new_g;
                    parent[nx][ny] = {x, y};
                    open_list.push(Node(nx, ny, new_g, h));
                    cout << " -> ADDED\n";
                } else {
                    cout << " -> SKIPPED\n";
                }
            }
        }
    }
    cout << "\nNo Path Found!\n";
}
// -------- MAIN --------
int main() {
    int n, m;
    cout << "Enter rows and columns: ";
    cin >> n >> m;
    vector<vector<int>> maze(n, vector<int>(m));
    cout << "Enter maze (0 = free, 1 = blocked):\n";
```

```
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> maze[i][j];
        }
    }
    int sx, sy, gx, gy;
    cout << "Enter start (x y): ";
    cin >> sx >> sy;
    cout << "Enter goal (x y): ";
    cin >> gx >> gy;
    astar(maze, {sx, sy}, {gx, gy});
    return 0;
}
```

```
PS C:\Users\parth\Desktop\AI\Assignment\Assignment 2> cd "c:\Users\parth\Desktop\AI\Assignment\Assignment 2\" ; if ($?) { g++ A2.cpp -o A2 } ; if ($?) { .\A2 }
Enter rows and columns: 5 5
Enter maze (0 = free, 1 = blocked):
0 0 0 0 0
1 1 0 1 0
0 0 0 1 0
0 1 1 0 0
0 0 0 0 0
Enter start (x y): 0 0
Enter goal (x y): 4 4

Current Node: (0,0) g=0 h=8 f=8
Checking (0,1) -> g=1 h=7 f=8 -> ADDED

Current Node: (0,1) g=1 h=7 f=8
Checking (0,0) -> g=2 h=8 f=10 -> SKIPPED
Checking (0,2) -> g=2 h=6 f=8 -> ADDED

Current Node: (0,2) g=2 h=6 f=8
Checking (1,2) -> g=3 h=5 f=8 -> ADDED
Checking (0,1) -> g=3 h=7 f=10 -> SKIPPED
Checking (0,3) -> g=3 h=5 f=8 -> ADDED

Current Node: (1,2) g=3 h=5 f=8
Checking (0,2) -> g=4 h=6 f=10 -> SKIPPED
Checking (2,2) -> g=4 h=4 f=8 -> ADDED

Current Node: (0,3) g=3 h=5 f=8
Checking (0,2) -> g=4 h=6 f=10 -> SKIPPED
Checking (0,4) -> g=4 h=4 f=8 -> ADDED

Current Node: (2,2) g=4 h=4 f=8
Checking (1,2) -> g=5 h=5 f=10 -> SKIPPED
Checking (2,1) -> g=5 h=5 f=10 -> ADDED

Current Node: (0,4) g=4 h=4 f=8
Checking (1,4) -> g=5 h=3 f=8 -> ADDED
Checking (0,3) -> g=5 h=5 f=10 -> SKIPPED

Current Node: (1,4) g=5 h=3 f=8
Checking (0,4) -> g=6 h=4 f=10 -> SKIPPED
```

```
PS C:\Users\parth\Desktop\AI\Assignment\Assignment 2> cd "c:\Users\parth\Desktop\AI\Assignment\Assignment 2\" ; if ($?) { g++ A2.cpp -o A2 } ; if ($?) { .\A2 }

Current Node: (1,4) g=5 h=3 f=8
Checking (0,4) -> g=6 h=4 f=10 -> SKIPPED
Checking (2,4) -> g=6 h=2 f=8 -> ADDED

Current Node: (2,4) g=6 h=2 f=8
Checking (1,4) -> g=7 h=3 f=10 -> SKIPPED
Checking (3,4) -> g=7 h=1 f=8 -> ADDED

Current Node: (3,4) g=7 h=1 f=8
Checking (2,4) -> g=8 h=2 f=10 -> SKIPPED
Checking (4,4) -> g=8 h=0 f=8 -> ADDED
Checking (3,3) -> g=8 h=2 f=10 -> ADDED

Current Node: (4,4) g=8 h=0 f=8

Goal Reached!

Final Path:
(0,0) (0,1) (0,2) (0,3) (0,4) (1,4) (2,4) (3,4) (4,4)
Path Length: 8
PS C:\Users\parth\Desktop\AI\Assignment\Assignment 2>
```